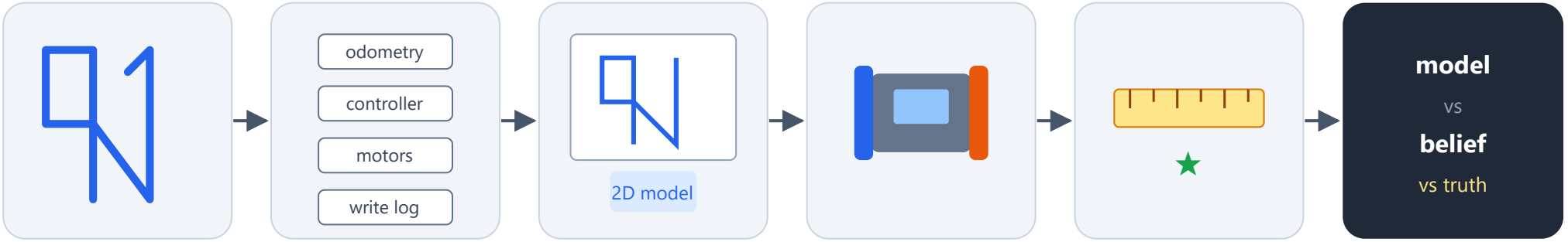
Localization with odometry

The robot draws my student-ID digits **9 1**, logs where it *thinks* it is, and we compare that with the simulator and a tape measure

The whole thing in one picture



1 · Plan "91"
7 straight legs, 7 turns

2 · Block program
one loop, every 20 ms

3 · Simulator
odometry_data_sim.txt

4 · Real robot
odometry_data_real.txt

5 · Tape measure
true end point

6 · Colab
Lab6_odometry.ipynb

Question
How well does a robot know where it is, if all it has is two wheel encoders?

Method
Same program in the simulator and on the robot. Both log odometry `x,y,time`. Compare the logs, then compare the robot's log with a tape measurement.

Why it matters
Every later lab (go-to-goal, gyro, Kalman filter) starts from this pose estimate and from its errors.

Result at a glance

real robot: odometry vs tape
12.4 cm
after 3.2 m of driving (≈ 4 %)

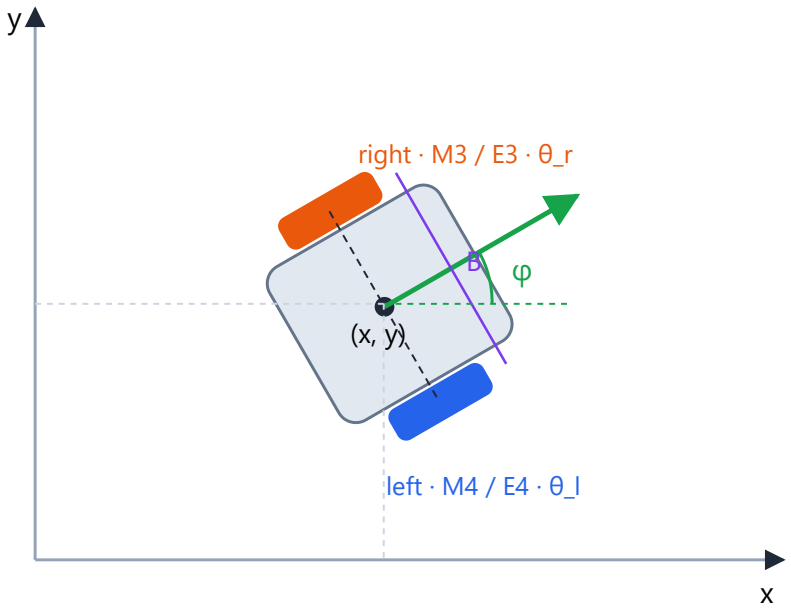
each leg longer than planned
5–13 cm
simulator: 0.2–1.3 cm

loop period
20 / 59 ms
simulator / real robot

simulator: odometry vs ground truth
1 mm
with the correct track B = 15.4 cm

1 Odometry: the model

dead reckoning from two encoders



The point the robot tracks is the **middle of the wheel axle**: in an in-place turn it stays where it is. Encoders count **degrees** of wheel rotation; both count up when driving forward (checked in the Lab 5 data).

The professor's form (used in the program, every loop pass)

$$\theta_l = \text{encoder4} \cdot \pi/180 \quad \theta_r = \text{encoder3} \cdot \pi/180$$

$$\text{path} = (\theta_r + \theta_l) \cdot r \,/\, 2$$

$$\varphi = (\theta_r - \theta_l) \cdot r \,/\, B$$

$$x \mathrel{+}= \Delta\text{path} \cdot \cos\varphi \quad y \mathrel{+}= \Delta\text{path} \cdot \sin\varphi$$

path how far the axle middle has rolled: the average of the two wheel arcs.

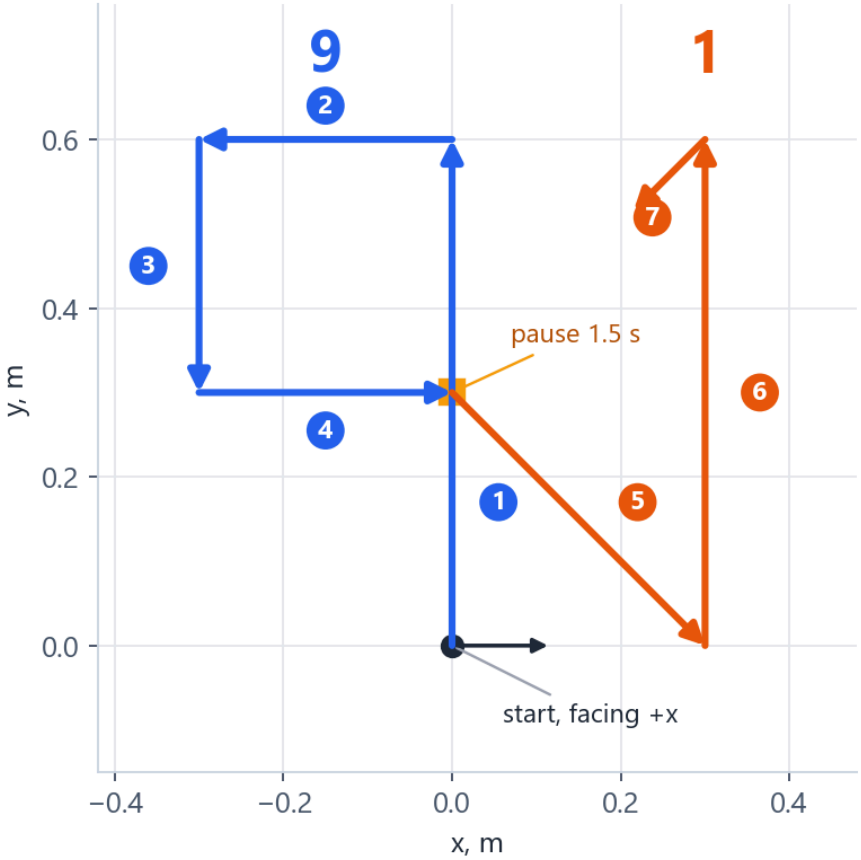
φ (course) heading. One wheel rolls further than the other → the robot has turned by the difference / B.

	R (WHEEL RADIUS)	B (TRACK)	LOG FILE
trajectory_sim.qrs	0.028 m	0.154 m	odometry_data_sim.txt
trajectory_real.qrs	0.0375 m	0.194 m	odometry_data_real.txt

Only r, B and the file name differ between the two programs: the simulated robot is physically smaller (16 px wheels, 44 px track) than the course robot. The real numbers are the professor's nominal values, not measured on my robot.

2 The trajectory: "9 1"

last two digits of the student ID



Drawn with straight legs and **in-place turns**, size a = 0.30 m, flag f = 0.12 m. The robot starts at the bottom of the 9's stem facing +x and first turns left. Both digits are one continuous path; leg 5 carries the robot from the 9 to the foot of the 1.

The plan is three lists in the `init` block (16 segments)

```
-- kind 0 = straight dist m, 1 = turn to heading hdg deg, 2 = pause dist ms
kind = {1, 0, 1, 0, 1, 0, 1, 0, 2, 1, 0, 1, 0, 1, 0, 2}
hdg  = {90, 90, 180, 180, 270, 270, 360, 360, 360,
        315, 315, 450, 450, 585, 585, 585}
dist = {0, 2*a, 0, a, 0, a, 0, a, 1500, 0, a*1.4142,
        0, 2*a, 0, f, 500}
```

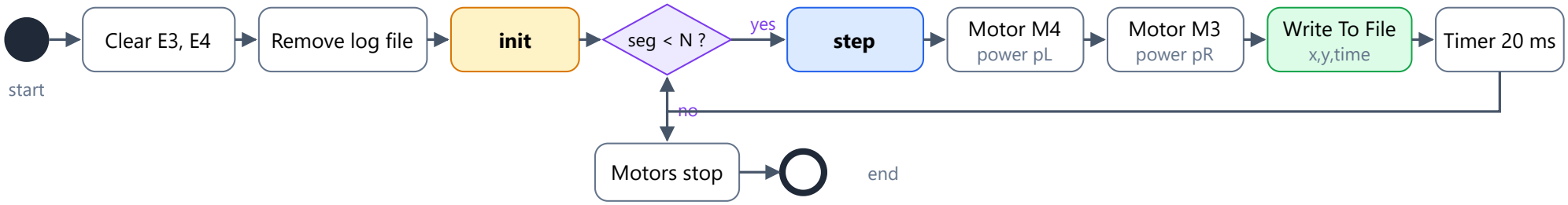
Absolute headings (90, 180, … 585°) instead of "turn 90°": a turn that stops 1° short is corrected by the next one, so heading errors in the *belief* don't add up.

Last segment = 0.5 s pause with motors off, so the final log lines show where the robot settled after coasting.

Total planned path: **2.64 m**. Robot run time: 37 s in the simulator, 23–28 s on the real robot.

3 The block program

trajectory_sim.qrs / trajectory_real.qrs




```
-- odometry: wheel angles in rad (left = E4, right = E3)
theta_l = encoder4 * pi / 180
theta_r = encoder3 * pi / 180
prev_path = path
path = (theta_r + theta_l) * r / 2
course = (theta_r - theta_l) * r / B
x = x + (path - prev_path) * cos(course)
y = y + (path - prev_path) * sin(course)
t = time() - t0
-- current segment: isS / isT / isP = 1 for straight / turn / pause
k = kind[seg]
isS = max(0, 1 - k);  isT = k * (2 - k);  isP = k * (k - 1) / 2
e = hdg[seg] * pi / 180 - course          -- heading error
rem = dist[seg] - (path - p0)             -- distance still to go
v = isS * min(VMAX, max(VMIN, KV * rem))
w = isS * KH * e + isT * sgn(e) * min(TMAX, max(TMIN, KT * abs(e)))
pL = v - w;  pR = v + w
-- segment finished? then start the next one from here
done = isS*max(0, sgn(-rem)) + isT*max(0, sgn(TOL - abs(e)))
       + isP*max(0, sgn(t - tp0 - dist[seg]))
done = min(1, done + (isS + isT) * max(0, sgn(t - tp0 - TMOUT)))
seg = seg + done;  p0 = p0 + done*(path - p0);  tp0 = tp0 + done*(t - tp0)
line = x .. "," .. y .. "," .. t .. "\n"
```

Function blocks have no `if`

So the segment type becomes a 0/1 factor. With k ∈ {0,1,2}:

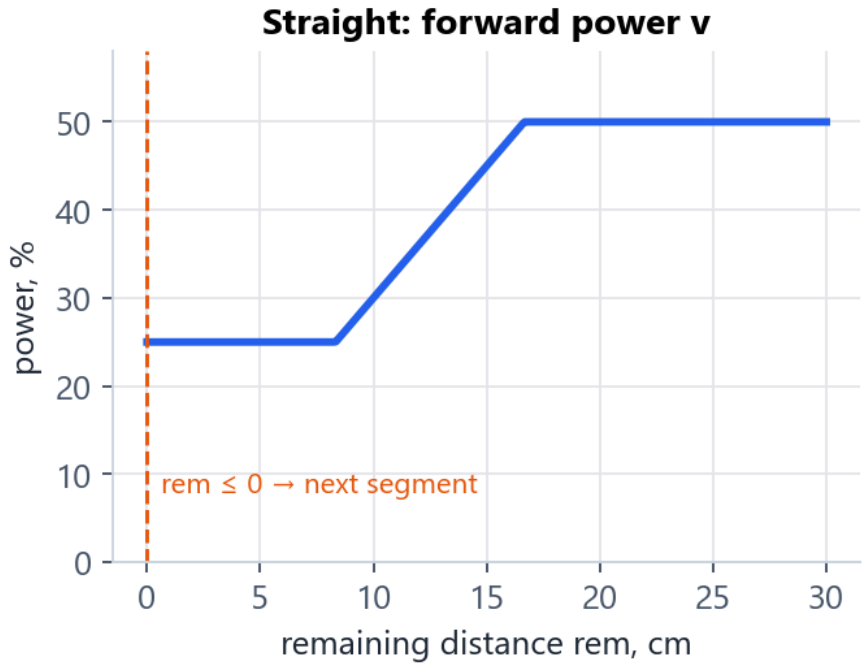
K	ISS	IST	ISP
0 straight	1	0	0
1 turn	0	1	0
2 pause	0	0	1

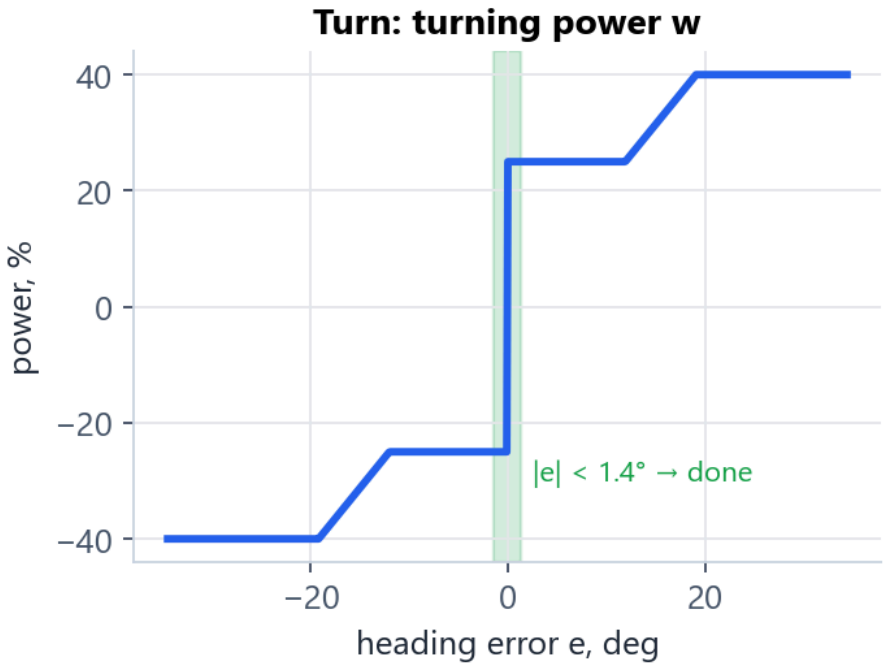
max(0, sgn(z)) is 1 when z > 0, else 0: a branch-free "if". When `done = 1`, `seg` moves on and `p0`, `tp0` remember where and when the new segment started.

The same blocks run on the robot: TRIK Studio compiles the diagram to JavaScript (`brick.encoder(E4).read()`, `script.writeToFile(...)`) on "Upload and run".

4 How it drives

the robot steers by its own odometry





Straight
Full speed (50 %) until 17 cm before the end, then slows down in proportion to the remaining distance, never below 25 %. Meanwhile `w = KH·e` keeps the heading: if the robot drifts 1°, the wheels get ±2.6 % power difference.

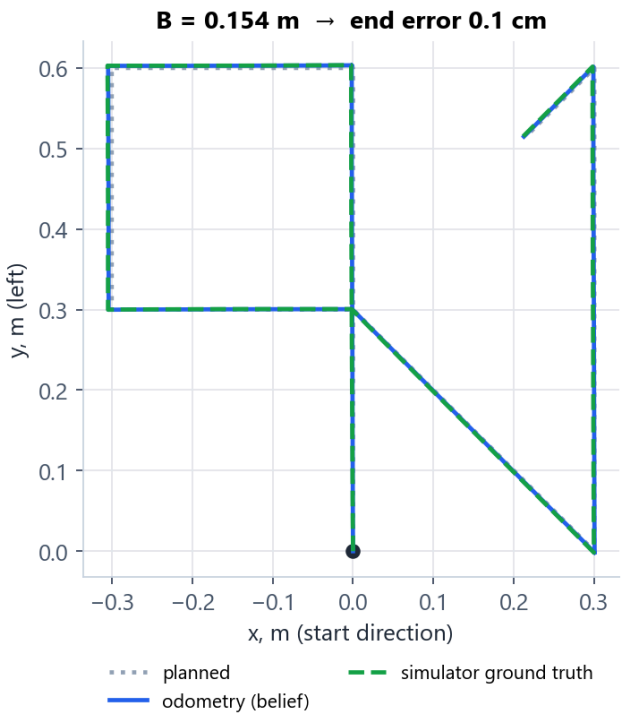
Turn in place
Wheels in opposite directions, power proportional to the heading error, between 25 % and 40 %. Done when the error is under 1.4°. The 25 % floor is there because the real robot does not move on the floor below about 10–15 % (Lab 5).

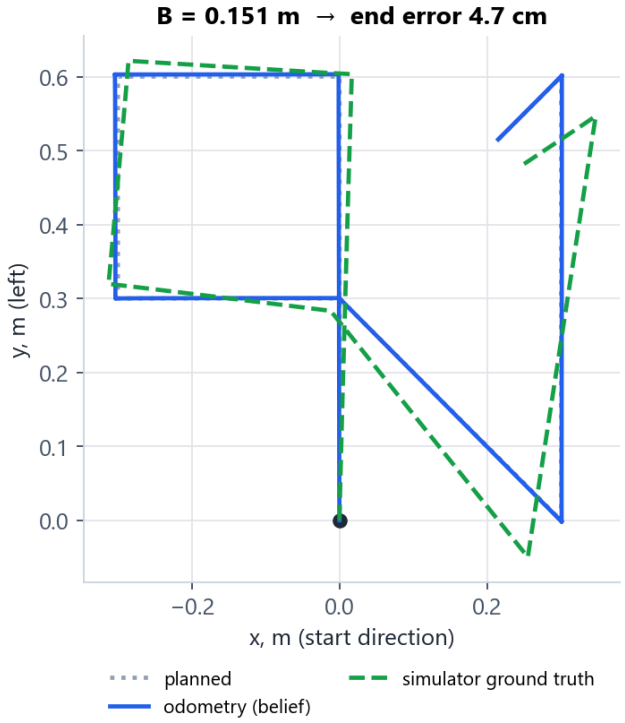
Safety
Every straight or turn gives up after 8 s, so the program always ends and the motors always stop, even if a turn keeps overshooting its tolerance.

Because the controller uses the odometry as feedback, the robot always drives to where it *believes* the next corner is. Its own log therefore looks close to the plan; only an outside measurement shows the real error.

5 Simulator check

the 2D model also records the true robot pose





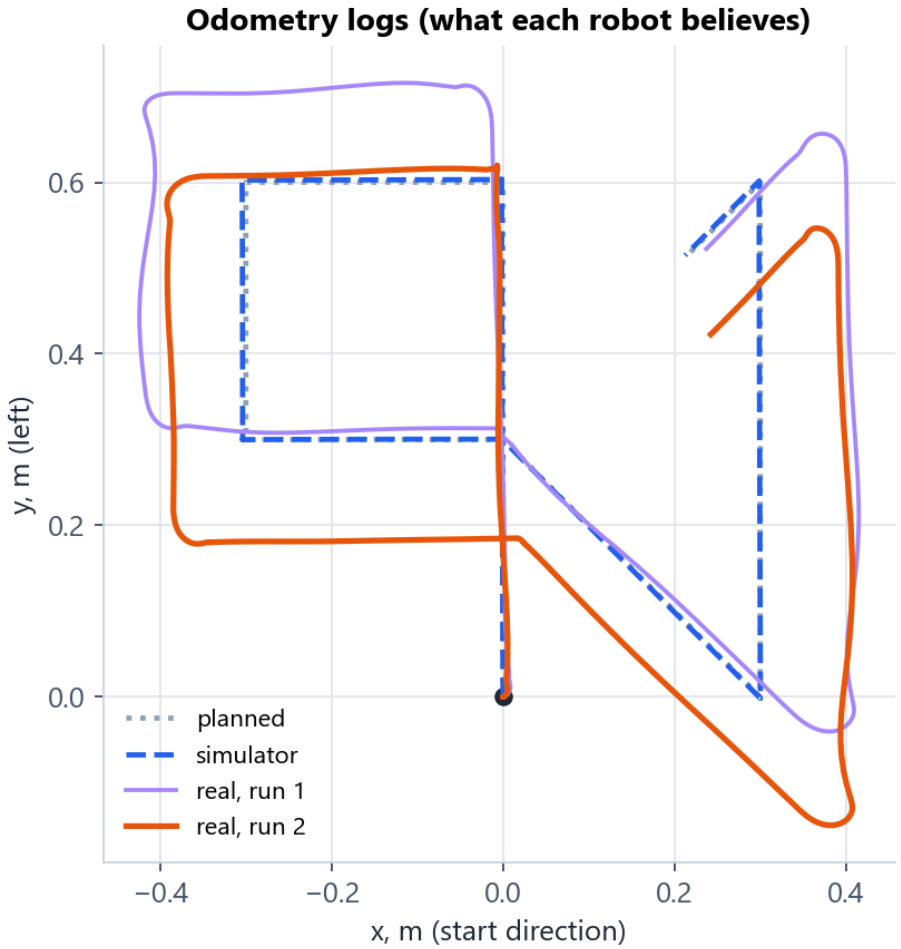
B = 0.154 m (44 px track)
Odometry and ground truth end **1 mm** apart. In the simulator the model is exact: no slip, ideal motors.

B = 0.151 m (2 % too small)
The robot believes it turned 585° but really turned only 573.6°. Every corner is about 1.6° short and the end point is **4.7 cm** off, while the log still looks perfect.

This is the lecture's "**wrong B**" error reproduced on purpose: a 2 % error in B gives a 2 % error in every turn, and it accumulates. The real robot's B (0.194 m) is the professor's number, not measured on my robot, so the same kind of error is expected there.

6 Real robot vs simulator

two runs of the same program, 22 Oct



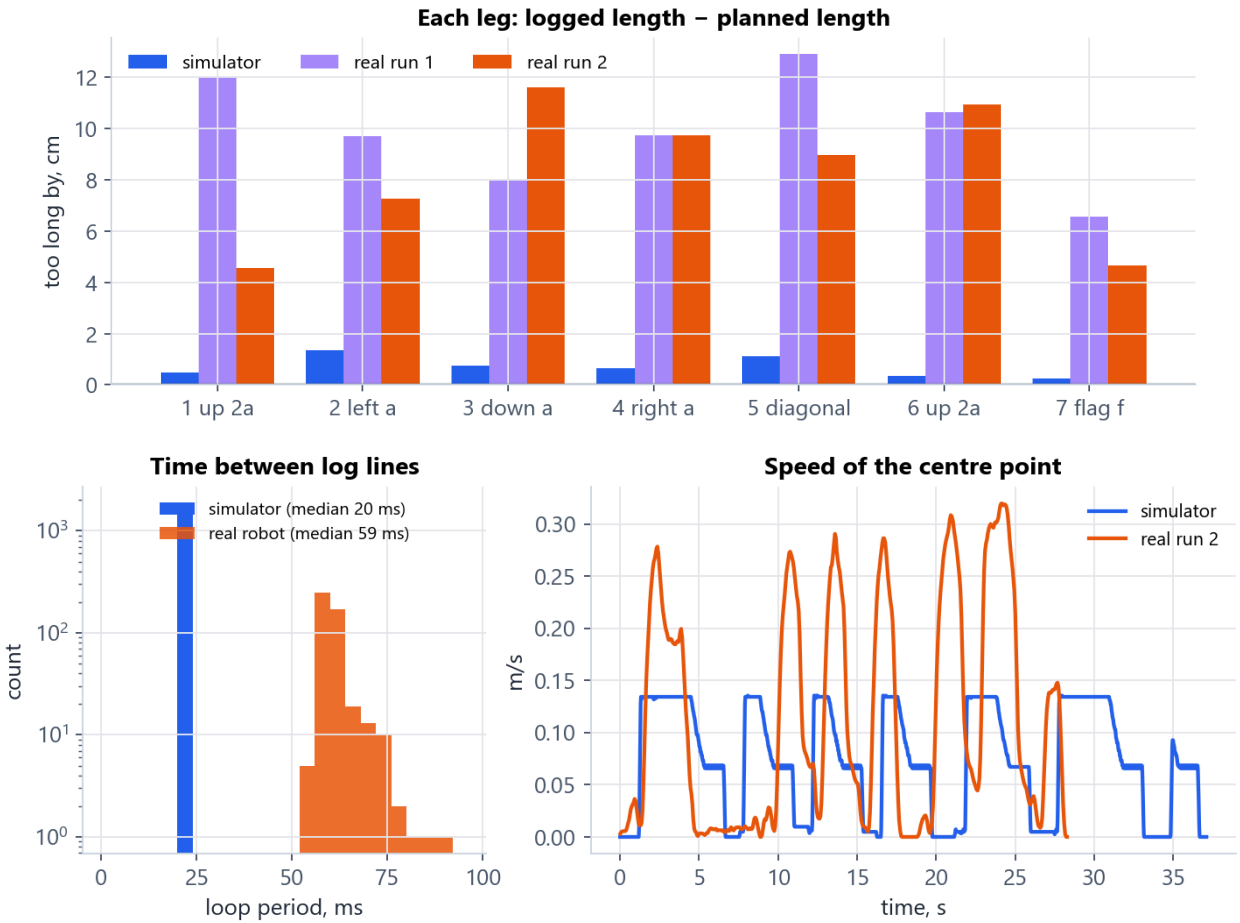
	TIME	PATH	LOG ENDS AT (X, Y)
planned	—	2.64 m	(0.215, 0.515)
simulator	37.1 s	2.69 m	(0.212, 0.515)
real, run 1	23.0 s	3.34 m	(0.237, 0.522)
real, run 2	28.3 s	3.22 m	(0.242, 0.423)

The real robot draws a recognisable "91", but **every leg is too long** and the corners are rounded, so the path is 20–25 % longer than planned. This is visible **in the robot's own log**: the robot knows it overshot.

The two real runs are **not the same**: same program, same robot, same floor, but the leg lengths differ by up to 5 cm and the end points by 10 cm.

The simulator log is almost exactly the plan: the 2D model has no motor lag and no dead zone, and its loop really runs every 20 ms.

7 Why the real legs are too long



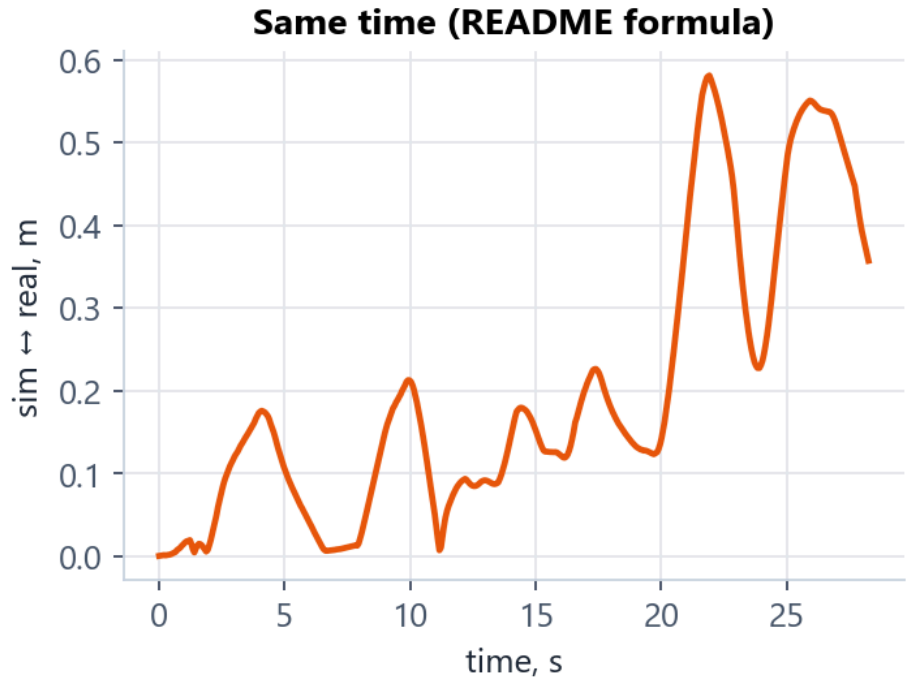
Slow loop. On the robot one pass took **59 ms** instead of 20 ms (the file write and the compiled JavaScript take time). The robot checks "distance reached?" only every ~2 cm at full speed.

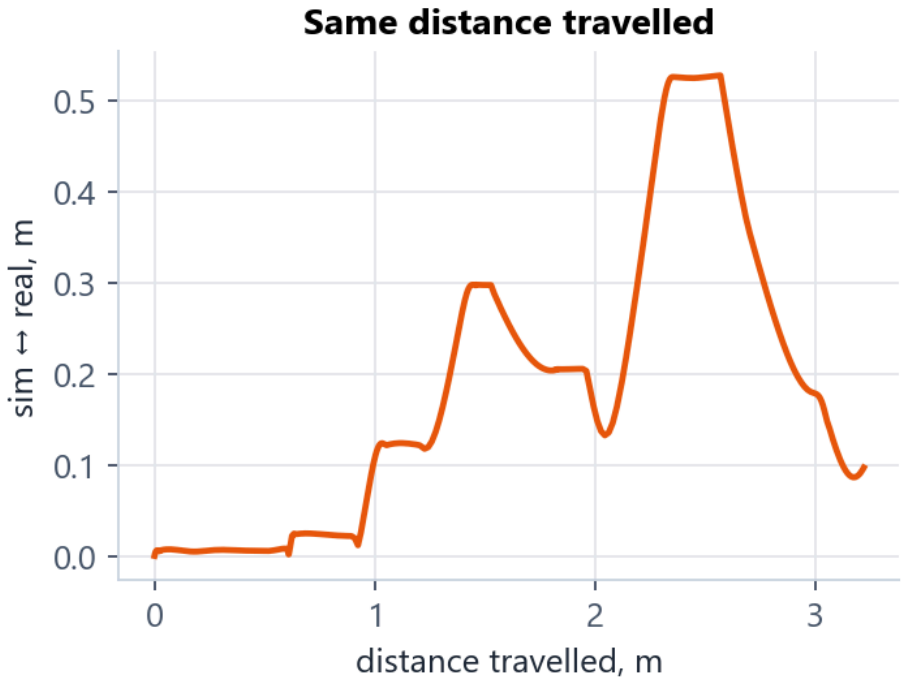
Fast + lagging motors. 50 % power is **0.32 m/s** on the robot (0.13 m/s in the simulator), and the motors need $T_m \approx 0.16$ s (Lab 5) to change speed. Coasting ≈ 0.32 m/s × (0.06 + 0.16) s ≈ **7 cm**, matching the 5–13 cm.

One slow turn. The centre point stands still in every in-place turn, but in run 2 the turn at 4.6 s took **3.9 s** instead of the usual 0.4–0.6 s (the flat part at 18 s is the planned 1.5 s pause). At 25 % turning power the robot probably stalled near its floor dead zone. Run 1 had no such turn: no two runs are alike.

8 Model vs real: the error curves

from the Colab notebook





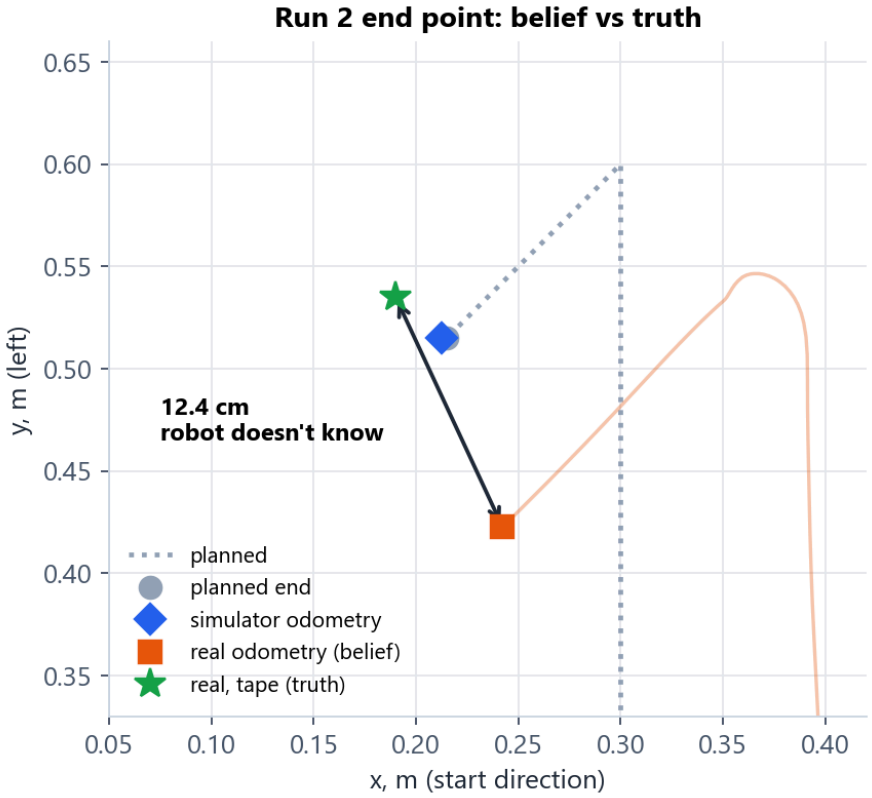
Same time (the README's formula)
Resamples the simulator onto the real timestamps. Max **0.58 m**. But the real robot drives 2.4× faster, so at the same time the two robots are on different legs. This curve mostly measures **speed**, not position.

Same distance travelled (added)
Compares points after the same arc length. Max **0.53 m**: every overshoot adds a few cm of extra path, so later in the run the real robot is "behind" in the drawing. It grows step by step with each leg.

End points
Simulator log vs real log at the end: **9.7 cm** apart (run 2). Small, because both robots steer back to the same believed corners.

9 Belief vs truth

run 2, measured with a tape



How it was measured
Before the run: tape mark under the middle of the axle (start S) and the start direction. After the run, without touching the robot: from S, **19 cm** along the start direction and **53.5 cm** to the left to the middle of the axle.

END POINT	X, M	Y, M
planned	0.215	0.515
simulator odometry	0.212	0.515
real odometry (last log line)	0.242	0.423
real, tape	**0.190**	**0.535**

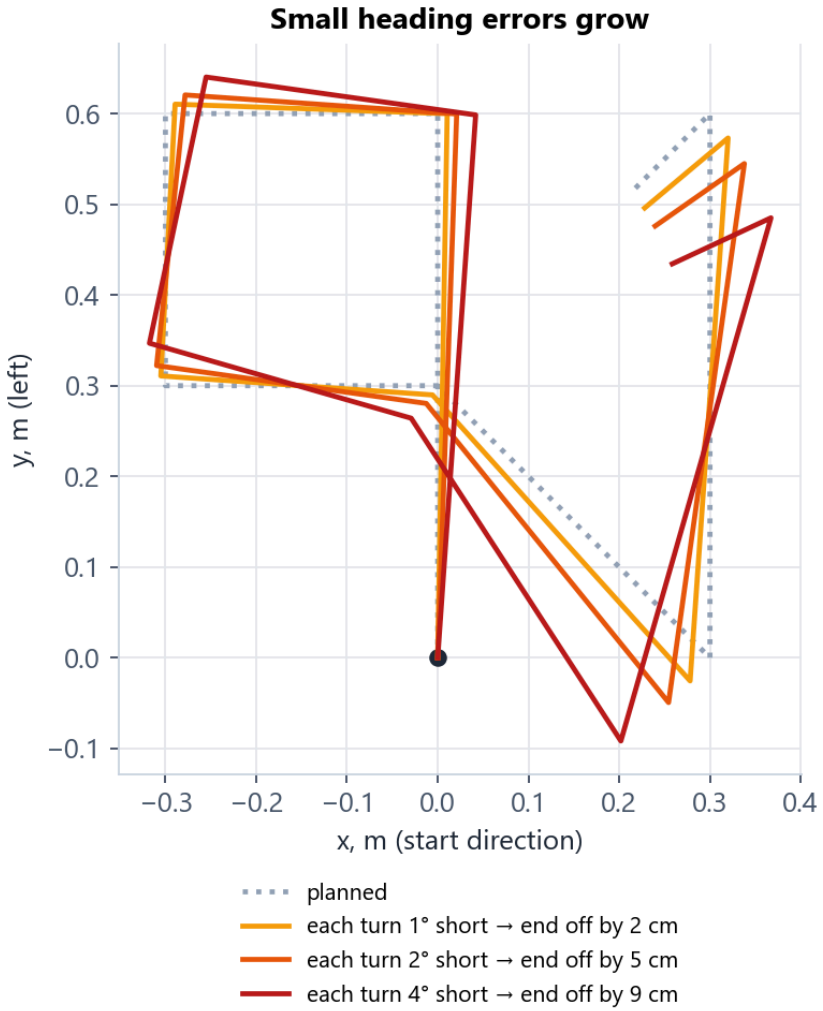
12.4 cm

between where the robot **thinks** it is and where it **is**, after 3.2 m: about 4 % of the distance, mostly sideways (11 cm in y).

Surprise: the real robot ended closer to the planned point (3 cm) than its own odometry thinks (10 cm). Probably part of what the encoders counted was not real motion (slip in the turns, nominal r and B), but one run cannot prove it: it could also partly be luck.

10 Which error sources?

the lecture's list vs the evidence



SOURCE	EVIDENCE IN THIS LAB
Heading error dominates	The belief-vs-tape error is mostly sideways. Replaying the plan with every turn 4° short already moves the end by 9 cm; ~12 cm needs a few degrees per turn.
Wheel slip	Seven in-place turns with the wheels scrubbing sideways; one turn in run 2 took 3.9 s instead of ~0.5 s, near the dead zone, where the wheels can slip on the floor.
Wrong r or B	r = 0.0375 and B = 0.194 are nominal (the professor's demo .qrs even uses r = 0.042). The simulator showed that 2 % in B alone gives 4.7 cm.
Unequal motors	Lab 5: k_e 1.03 vs 1.22 V·s/rad. Each encoder measures its own wheel, so the odometry sees the speed difference and the heading hold corrects it. What it cannot see is unequal wheel *radii* (not measured).
Timing / dynamics (not in the list)	59 ms loop + 0.16 s motor lag → 5–13 cm overshoot per leg. The robot knows about this one: it is in the log.

> **Does the robot know it is wrong?** No. Its log ends at a confident (0.242, 0.423); nothing on board can tell it that it is really at (0.19, 0.535).

11 Summary

Five takeaways

1 · Odometry integrates two encoders into (x, y, φ). In the simulator it is exact (1 mm after 2.7 m).

2 · A 2 % error in the track B turns every corner 2 % wrong. The log does not show it.

3 · On the real robot every leg overshoots 5–13 cm: 59 ms loop + motor lag + coasting. The robot sees this in its own log.

4 · Odometry vs tape: 12.4 cm after 3.2 m (≈ 4 %), mostly a heading error from slip and nominal r/B.

5 · Two identical runs end 10 cm apart, and the robot has no way to notice: encoders alone cannot detect their own errors.

What I would do next

Measure r and B: drive 1 m straight and compare with the tape (→ r); spin 10 full turns in place and compare (→ B).

Less overshoot: log every few passes (faster loop), brake earlier using the measured speed, ramp the speed up and down.

Better heading: a gyroscope (Week 10) measures rotation directly, without wheel slip: the biggest error source here.

Files: trajectory_sim.qrs, trajectory_real.qrs, odometry_data_sim.txt, odometry_data_real.txt (+ run 1), Lab6_odometry.ipynb, README.md in `06_Localization/Lab6/`.